

BillieBot — System Design Document (MVP v0.1)

Role: Senior Systems Engineering deliverable **Scope:** Requirements analysis, tiered System of Systems (SoS) model for Cameo/MSOSA, and hardware/software implementation design for the BillieBot MVP, with explicit hooks for the future Behavior AI and learning loop.

1. Concept of Operations (ConOps) Summary

BillieBot is a differential-drive indoor robot that autonomously patrols an apartment, locates a miniature dachshund (Billie), observes her from a non-intrusive standoff distance, classifies her behavioral state from fused visual/thermal/audio evidence, logs those states as timestamped events, and produces a daily activity summary for the owner. The MVP is **observe-and-report only**; it never initiates engagement. The architecture, however, is deliberately partitioned so that a future *Behavior Policy* subsystem (contextual bandit / RL-lite) can be dropped in as a decision node that consumes the same state estimates the MVP already produces and commands the same action primitives the MVP already implements for its own use (navigate, approach, retreat, speak).

Operational modes: IDLE/DOCKED, PATROL, INVESTIGATE (audio-cued), TRACK/OBSERVE, RETURN, SAFE (low battery / fault).

2. Requirements

2.1 Stakeholder (Level 0) Requirements

ID	Text (shall statement)	Source
STK-1	BillieBot shall autonomously navigate the apartment without getting stuck.	User req (i)
STK-2	BillieBot shall autonomously find Billie.	User req (ii)
STK-3	BillieBot shall visually detect Billie using a visible-light and/or IR camera.	User req (iii)
STK-4	BillieBot shall auditorily detect when Billie barks or makes other loud noises.	User req (iv)
STK-5	BillieBot shall classify and log Billie's state (e.g., sleeping, active, barking).	User req (v)

ID	Text (shall statement)	Source
STK-6	BillieBot shall generate a daily summary of Billie's activities.	User req (vi)
STK-7 (<i>post-MVP</i>)	BillieBot shall decide when/if to engage Billie (approach, retreat, speak, dispense treat).	User future req
STK-8 (<i>post-MVP</i>)	BillieBot shall learn which actions increase engagement without causing stress (contextual bandit / RL-lite).	User future req

2.2 System (Level 1) Requirements — derived, verifiable

Each L1 requirement traces to ($\leftarrow$ *deriveReq*) an L0 requirement and carries a verification method: Inspection, Analysis, Demonstration, Test.

Navigation (SYS-NAV) $\leftarrow$ STK-1, STK-2

ID	Requirement	Verify
SYS-NAV-1	The system shall build and persist a 2-D occupancy map of the apartment using onboard lidar SLAM.	D
SYS-NAV-2	The system shall localize within the saved map with ≤ 0.15 m mean position error.	T
SYS-NAV-3	The system shall plan and execute collision-free paths between any two reachable waypoints, replanning around dynamic obstacles (incl. the dog).	D
SYS-NAV-4	The system shall detect a stuck condition (commanded motion, no odometric progress for > 5 s) and execute recovery behaviors (back-up, spin, replan); after 3 failed recoveries it shall enter SAFE mode and alert the operator.	T
SYS-NAV-5	The system shall limit speed to ≤ 0.3 m/s in normal transit and ≤ 0.15 m/s within 2 m of a detected dog.	T
SYS-NAV-6	The system shall execute a configurable patrol route over user-defined waypoints covering all rooms.	D

Search / Find (SYS-FND) $\leftarrow$ STK-2

ID	Requirement	Verify
SYS-FND-1	The system shall search patrol waypoints in priority order (last-known location first, then likelihood-weighted) until the dog is detected or all waypoints are exhausted.	D
SYS-FND-2	Upon an audio event with direction-of-arrival (DoA), the system shall re-prioritize search toward the DoA bearing.	D
SYS-FND-3	Upon visual detection, the system shall approach to and hold a standoff distance of 1.0–2.0 m (never closer than 1.0 m in MVP).	T
SYS-FND-4	The system shall find the dog within 10 minutes in $\geq 80\%$ of trials when the dog is in a mapped room.	T (acceptance)

Perception (SYS-PER) $\leftarrow$ STK-3, STK-4

ID	Requirement	Verify
SYS-PER-1	The system shall detect a dog in RGB imagery at ≥ 5 FPS with $\geq 85\%$ recall at ≤ 4 m range (COCO class dog , tuned threshold).	T
SYS-PER-2	The system shall estimate the 3-D position of a detected dog in the map frame using stereo depth (± 0.2 m at 2 m).	T
SYS-PER-3	The system shall detect a warm body ($30\text{--}40^\circ\text{C}$ blob $\geq N$ pixels) with the thermal camera in darkness at ≤ 1.5 m.	T
SYS-PER-4	The system shall detect bark/whine/howl/loud-noise audio events with onboard classification ($\geq 80\%$ recall on a recorded Billie test set) and report DoA ($\pm 15^\circ$).	T
SYS-PER-5	The system shall low-light detect the dog via the NoIR camera when ambient lux $<$ threshold (requires IR illuminator — see HW gap list).	D

State Estimation & Logging (SYS-STL) $\leftarrow$ STK-5

ID	Requirement	Verify
SYS-STL-1	The system shall fuse visual, thermal, and audio evidence into a dog-state estimate from the set {SLEEPING, RESTING, ACTIVE, BARKING, EATING/AT-BOWL*, NOT-FOUND} with confidence. (*optional, waypoint-based)	T
SYS-STL-2	The system shall log every state transition and detection event to a persistent store (SQLite) with timestamp, state, confidence, map location, and an image snapshot reference.	T
SYS-STL-3	The state message schema shall be forward-compatible with the Behavior AI (include feature vector fields reserved for the bandit context).	I
SYS-STL-4	Logging shall survive power loss without database corruption (WAL mode; snapshot fsync).	T

Reporting (SYS-RPT) ← STK-6

ID	Requirement	Verify
SYS-RPT-1	The system shall generate, once per day at a configured time, a summary containing: total sleep/rest/active durations, bark count and times, activity timeline, rooms visited by dog (from detection locations), and representative snapshots.	D
SYS-RPT-2	The summary shall be retrievable from the host computer via the local network (HTTP page or file share).	D

Platform / Support (SYS-PLT)

ID	Requirement	Verify
SYS-PLT-1	Endurance ≥ 60 min continuous patrol per battery charge.	T
SYS-PLT-2	The system shall monitor battery voltage and enter SAFE mode (stop, alert, request pickup) at ≤ 3.5 V/cell; hard cutoff behavior documented at 3.3 V/cell.	T
SYS-PLT-3	All power branches shall be individually fused; motor and compute rails shall be electrically separated at the distribution bus.	I
SYS-PLT-4	The operator shall be able to teleoperate, visualize (map, camera, detections), configure waypoints, and e-stop from the host computer over Wi-Fi.	D
SYS-PLT-5	A software e-stop shall cut motor PWM within 200 ms; loss of the Jetson $\leftrightarrow$ Arduino serial heartbeat > 500 ms shall stop the motors autonomously (Arduino-side watchdog).	T
SYS-PLT-6	Acoustic design: no sudden loud sounds in MVP; speaker output reserved and rate-limited (dog-welfare constraint supporting future no-stress learning).	I

Extensibility (SYS-EXT) $\leftarrow$ STK-7, STK-8 (design constraints on the MVP, not MVP features)

ID	Requirement	Verify
SYS-EXT-1	Engagement primitives (ApproachDog, Retreat, Speak, DispenseTreat-stub) shall be implemented as ROS 2 action servers with a uniform interface, even though the MVP mission logic only invokes ApproachDog/Retreat internally.	I
SYS-EXT-2	Mission logic shall be a behavior tree with a designated <code>PolicyDecision</code> extension point; the MVP plugs in a static <code>ObserveOnlyPolicy</code> .	I
SYS-EXT-3	The event log shall record (context, action, outcome) tuples for all robot actions near the dog, so historical data can bootstrap the future bandit.	I
SYS-EXT-4	The dog-state message shall include a <code>stress_proxy</code> field (MVP: heuristic from bark rate + retreat motion; future: learned).	I
SYS-EXT-5	Power distribution and one L298N channel-equivalent spare (or 5 V rail headroom ≥ 2 A) shall be reserved for a treat-dispenser actuator.	I

2.3 Requirements Diagram (Cameo **req** diagram content)

Create package **1 Requirements** with two diagrams:

- **REQ-01 "Stakeholder Requirements"** — STK-1...STK-8 as **«requirement»** elements; STK-7/STK-8 stereotyped additionally as **«futureRelease»** (create this stereotype in a small profile, or tag with a **release = "v2"** custom property).
- **REQ-02 "System Requirements Trace"** — SYS-* requirements grouped in **«requirement»** containers per category (NAV, FND, PER, STL, RPT, PLT, EXT), with:
 - **deriveReq** dependencies from SYS-* to STK-*,
 - **satisfy** dependencies from blocks (Section 3) to SYS-*,
 - **verify** dependencies from test cases (name them TC-NAV-4, TC-FND-4, TC-PER-1 ... matching Section 6 test plan).

3. System of Systems Model (Cameo/MSOSA)

3.1 Model organization (containment tree)

```
BillieBot SoS Model
├─ 0 Profiles                («futureRelease», «rosNode», «rosTopic»
stereotypes)
├─ 1 Requirements            (REQ-01, REQ-02)
├─ 2 Use Cases               (UC diagram: Patrol, Find Dog, Log State, Daily
Report,
                             Teleoperate, E-Stop, [future] Engage Dog, Learn
Policy)
├─ 3 Structure
│   ├─ 3.1 SoS Context        (BDD-00, IBD-00)
│   ├─ 3.2 Robot Segment      (BDD-01 ... BDD-06, IBD-01 ... IBD-03)
│   └─ 3.3 Support Segment    (router, host computer, charger)
├─ 4 Behavior                (ACT-01 ... ACT-04, STM-01 mission state machine)
├─ 5 Interfaces              (InterfaceBlocks, Signals, ItemFlows, message
defs)
└─ 6 Analysis                (allocation matrices: SW→HW, Req→Block; power
rollup)
```

3.2 Tier 0 — SoS Context

BDD-00 "BillieBot System of Systems" — blocks and associations:

Block	Kind	Notes
<u>BillieBot SoS</u>	system-of-systems block	composes the three segments below
<u>Robot Segment :: BillieBot Rover</u>	constituent system	the mobile robot
<u>Support Segment :: WiFi Router (GL-SFT1200)</u>	constituent system	dedicated robot WLAN, stationary in apartment
<u>Support Segment :: Host Computer</u>	constituent system	operator UI (RViz/Foxglove), report viewer, dev workstation
<u>Support Segment :: Battery Charger</u>	constituent system	offboard LiPo balance charger (manual swap/charge in MVP)
<u>Apartment Environment</u>	external / environment block	rooms, furniture, floor surfaces, lighting
<u>Billie (Dog)</u>	external actor block	the subject of observation; value props: mass $\approx$ 5 kg, height $\approx$ 0.15-0.25 m
<u>Operator (Owner)</u>	actor	

Associations: Rover —(Wi-Fi 802.11)— Router —(Wi-Fi/Ethernet)— Host; Rover —(senses)→ Billie; Rover —(navigates within)→ Apartment.

IBD-00 — connectors typed by interface blocks: IF_WiFi_DDS (ROS 2/DDS traffic), IF_HTTP_Report, IF_SSH_Admin; item flows: MapData, Telemetry, Video, DogEvents, DailySummary, TeleopCmd.

3.3 Tier 1 — Robot Segment decomposition

BDD-01 "BillieBot Rover" — the rover block composes seven subsystems (part properties):

Part	Subsystem block	Primary satisfy
<u>mob : Mobility Subsystem</u>	drive + low-level control	SYS-NAV-3/4/5, SYS-PLT-5
<u>nav : Navigation & Autonomy Subsystem</u> (software subsystem hosted on compute)	SLAM, localization, planning, mission BT	SYS-NAV-, SYS-FND-
<u>per : Perception Subsystem</u>	cameras + detectors	SYS-PER-1/2/3/5
<u>aud : Audio Subsystem</u>	mic array, classifier, speaker	SYS-PER-4, SYS-PLT-6
<u>cog : Cognition & Logging Subsystem</u> (software)	state fusion, event log, reporting, policy extension point	SYS-STL-, SYS-RPT-, SYS-EXT-*
<u>cmp : Compute & Comms Subsystem</u>	Jetson, Pi 5, Arduino, network	hosts nav/per/aud/cog software
<u>pwr : Power Subsystem</u>	battery, regulation, distribution, protection	SYS-PLT-1/2/3
<u>str : Structure/Chassis</u>	frame, mounts, wiring harness	—

3.4 Tier 2 — Subsystem BDDs (hardware allocation)

BDD-02 Mobility Subsystem

Part	Component	Key value properties
<code>mtrL, mtrR : JGA25-371 Gearmotor</code>	12 V, 130 rpm, integrated quadrature encoder	stall $\approx$ 1.2–2 A ea
<code>whlL, whlR : Wheel Ø68 mm</code>	rubber	$v_{\text{max}} \approx 0.46$ m/s @130 rpm
<code>cstr : Caster Wheel</code>	rear (or front) third contact	
<code>hbridge : L298N Dual H-Bridge</code>	motor driver	~ 2 V drop; heatsink
<code>mcu : Arduino Nano V3</code>	encoder counting, PID velocity loop, PWM, IMU host, battery ADC, heartbeat watchdog	ATmega328P, 16 MHz
<code>imu : BN0055 + BMP280 (DFRobot 10-D0F)</code>	fused orientation (I ² C on the Arduino — avoids Pi/Jetson I ² C clock-stretching problems)	100 Hz quat + gyro

BDD-03 Perception Subsystem

Part	Component	Interface / host	Role
<code>oakd : Luxonis OAK-D Lite</code>	USB-3 $\rightarrow$ Jetson	Primary detector: on-device YOLOv8n (RVC2) + stereo depth $\rightarrow$ 3-D dog position	
<code>lidar : RPLidar A1</code>	USB(UART) $\rightarrow$ Jetson	2-D SLAM + costmaps (grouped here or under Nav — model it under Perception, allocate data to Nav)	
<code>noir : Pi Camera Module 3 NoIR</code>	CSI-0 $\rightarrow$ Pi 5	Low-light imagery (gap : add 850 nm IR illuminator)	
<code>thermal : MLX90640 (55°)</code>	I ² C $\rightarrow$ Pi 5	Warm-blob presence/sleep confirmation, works in total darkness	
<code>rgb2 : Pi Camera v2</code>	CSI-1 $\rightarrow$ Pi 5 (optional)	Spare / rear view / dock-cam; not required for MVP	

BDD-04 Audio Subsystem

Part	Component	Interface / host	Role
<u>micarr : ReSpeaker</u> <u>XVF3800 4-mic array</u>	USB → Pi 5	Beamformed audio + direction of arrival	
<u>amp : MAX98357A I²S amp</u> + <u>spk : 3 W speaker</u>	I ² S GPIO → Pi 5	Status chimes only in MVP; <u>Speak</u> action reserved for Behavior AI	
<u>mems : SPH0645 I²S mic</u>	(spare)	Backup mic if XVF3800 driver issues arise	

BDD-05 Compute & Comms Subsystem

Part	Component	Role
<u>jet : Jetson Orin</u> <u>Nano Super (+500 GB</u> <u>NVMe)</u>	Primary autonomy computer. ROS 2, slam_toolbox/AMCL, Nav2, DepthAI pipeline, mission BT, base-bridge serial node	
<u>pi : Raspberry Pi 5</u> <u>(4 GB) + USB hub</u>	Sensing/cognition companion. Audio classifier + DoA node, thermal node, NoIR camera node, event logger (SQLite), daily-report generator, web report server	
<u>mcu</u>	(shared part, see Mobility) real-time I/O	
<u>rtr : GL-SFT1200</u>	modeled in Support Segment; reference association here	

BDD-06 Power Subsystem — see Section 4.2 for the electrical design this models.

Part	Component
bat : Venom 3S LiPo 11.1 V 4000 mAh 20C	
sw : Master switch, fMain/fMot/fCmp : Fuses, bus : Terminal strip/bus	
regJet : (direct 3S feed, 9–19 V input) — Jetson barrel jack from protected bus	
regPi : GeeekPi PD board (12 V→5 V/5 A USB-C PD) → Pi 5	
reg5A : DFRobot GS2678 buck → 5 V accessory rail (Arduino, amp, IR illuminator, fans)	
reg9A : Pololu D24V90F5 5 V/9 A → reserved rail (future treat dispenser / actuator per SYS-EXT-5)	
hbridge	(shared part) 12 V motor branch
vsense : Divider → Arduino ADC	battery monitor

3.5 Internal Block Diagrams

IBD-01 "Rover — Data & Control" (ports typed by interface blocks; connectors carry item flows):

```

lidar —USB/UART→ jet          oakd —USB3→ jet
mcu ←USB-serial 115200→ jet    (frames: cmd_vel dn / ticks,IMU,battV up @50
Hz)
imu  —I2C→ mcu                encoders —quadrature→ mcu
mcu  —PWM+DIR→ hbridge —12V drive→ mtrL,mtrR
thermal —I2C→ pi              noir —CSI→ pi
micarr —USB→ pi               pi —I2S→ amp → spk
jet ←WiFi/DDS→ rtr ←WiFi→ pi   rtr ←WiFi→ host

```

Item flows to define: LaserScan, StereoFrames+Detections, WheelCmd, WheelTicks, ImuSample, BattVoltage, ThermalFrame, AudioEvent+DoA, DogState, MissionStatus, TeleopCmd.

IBD-02 "Rover — Power Distribution" (connectors carry ItemFlow : ElectricalPower with voltage/current properties):

```

bat → fMain(20A) → sw → BUS(11.1 V nom)
BUS → fMot(10A) → hbridge → motors (motor branch)
BUS → fCmp(10A) → jet (barrel, 9-19 V direct) (compute branch)
BUS → fCmp → regPi(PD 5V/5A) → pi(USB-C)
BUS → fAux(5A) → reg5A → 5 V accessory rail → mcu, amp, IR LED
BUS → (reserved) → reg9A → future actuator rail
BUS → vsense → mcu ADC

```

IBD-03 "Software Deployment / ROS Graph" — parts are «rosNode» blocks allocated to (jet)/(pi)/(mcu) execution environments; connectors are «rosTopic»/service/action flows (see Section 5.2 table — reproduce it as the IBD).

3.6 Behavior — Activity Diagrams & State Machine

STM-01 "Mission Modes" (state machine on the Rover block): DOCKED/IDLE → PATROL → (audio event) INVESTIGATE → TRACK/OBSERVE → PATROL, with global transitions to SAFE (battery/fault/e-stop) and RETURN (schedule end). The TRACK/OBSERVE → ENGAGE transition exists in the model, guarded [policy ≠ ObserveOnly], stereotyped «futureRelease» — this is the documented Behavior-AI insertion point.

ACT-01 "Patrol & Find Billie" (swimlanes: Navigation | Perception | Audio | Mission BT)

1. LoadMap → Localize(AMCL) → decision [localized?]
2. SelectNextWaypoint(priorityQueue) — queue seeded by last-known dog location; **interruptible signal** AudioEvent(DoA) re-sorts queue (accept-event action).
3. Nav2.NavigateToPose(wp) → fork: ScanRGB(YOLO dog) // ScanThermal (both continuous object flows into DetectionFusion).
4. Decision [dog detected] → ComputeStandoffPose(1.5 m) → Nav2.NavigateToPose(standoff) → send signal DogFound → to ACT-02.
5. [else] loop to step 2; [queue empty] → MarkNotFound → wait/retry timer.

ACT-02 "Observe, Classify & Log" (swimlanes: Perception | Audio | Cognition)

1. Parallel streams: RGBDetections(bbox, depth, motion), ThermalBlob(size, T), AudioEvents(class, level, DoA) → StateFusion (sliding 10 s window, hysteresis).
2. ClassifyState → {SLEEPING (blob static, low aspect, no motion, no audio) | RESTING | ACTIVE (motion energy) | BARKING (audio class)}.
3. Decision [state changed] → SnapshotImage → WriteEvent(SQLite: t, state, conf, pose, img, contextVector) (contextVector satisfies SYS-EXT-3).
4. PublishDogState (topic consumed by mission BT; future consumer: Behavior Policy).

ACT-03 "Generate Daily Summary" (swimlanes: Scheduler | Cognition | Support) (23:30 timer) → (QueryEvents(day)) → (Aggregate(sleep %, active %, bark count/times, room histogram, timeline)) → (RenderReport(Markdown+HTML, embed snapshots)) → (PublishToWebServer) + (WriteToShare) → (NotifyOperator).

ACT-04 "Stuck Detection & Recovery" (Navigation lane) (MonitorProgress(cmd vs odom)) → ([no progress > 5 s]) → (Recovery1: back-up 0.2 m) → (Recovery2: rotate ±45°) → (Replan) → ([3 failures]) → (SAFE + alert) (satisfies SYS-NAV-4).

(For Cameo: model interruptible regions around the patrol loop in ACT-01, use accept-event actions for (AudioEvent) and (BatteryLow) signals, and allocate activity partitions to the subsystem blocks so the allocation matrix auto-populates.)

3.7 Allocation matrices (package 6)

- **Req → Block** satisfy matrix (rows SYS-*, columns subsystem blocks).
- **SW node → HW** allocation matrix (rows = «rosNode» blocks in IBD-03, columns = jet/pi/mcu).
- **Power rollup** table (can be a simple instance table; parametrics optional later).

4. Hardware Design

4.1 Compute & interface allocation (rationale)

Host	Owns	Why
Jetson Orin Nano Super	RPLidar A1 (USB), OAK-D Lite (USB-3), Arduino serial link, ROS 2 core autonomy (SLAM/Nav2/mission BT), YOLO post-processing	GPU + USB-3 bandwidth; keeps the perception→planning loop on one machine (no Wi-Fi in the control loop)
Raspberry Pi 5	ReSpeaker XVF3800 (USB via hub), MLX90640 (I²C), Pi Cam 3 NoIR (CSI-0), Pi Cam v2 (CSI-1, optional), MAX98357A (I²S), SQLite event log, report generator + web server	Pi 5 has dual CSI, good I²C/I²S GPIO; isolates bursty audio/thermal/logging work from the real-time nav stack
Arduino Nano V3	Encoders (D2/D3 interrupts + pin-change for B channels), L298N PWM/DIR, BNO055 I²C, battery ADC, 500 ms heartbeat motor cutoff	True real-time PWM/encoder handling; BNO055 clock-stretching is unreliable on Pi/Jetson hardware I²C but fine on AVR

Notes and gotchas:

- **OAK-D Lite on USB-3:** budget ≈ 4.5 W; if you see brownout resets, feed it through a powered hub or Y-cable from the 5 V accessory rail.
- **Nano encoder inputs:** only D2/D3 are true external interrupts. Use D2/D3 for channel A of each encoder and pin-change interrupts (or $\times 2$ decoding) for channel B — plenty at 130 rpm.
- **IR illumination gap:** the NoIR camera has no emitter. Add an 850 nm LED board (~ 1 –3 W) on the 5 V accessory rail, switched by an Arduino GPIO + MOSFET, or rely on the thermal camera at night for MVP.
- **L298N:** lossy (~ 2 V drop) but adequate; plan a TB6612FNG/Cytron swap in v2 for efficiency and cooler running.

4.2 Power architecture

```
3S LiPo 11.1 V 4 Ah —[20 A main fuse]—[master switch]—► DISTRIBUTION BUS
BUS —[10 A]—► L298N VMOT —► motors (2× JGA25-371)
BUS —[10 A]—► Jetson DC barrel (accepts 9–19 V; 3S range 9.9–12.6 V ✓)
BUS —[ 5 A]—► GeekPi PD board —USB-C PD 5 V/5 A—► Pi 5
BUS —[ 5 A]—► DFRobot GS2678 buck @5 V —► Arduino VIN(5V pin),
MAX98357A, IR LED, fan
BUS —(reserved, fused)—► Pololu D24V90F5 5 V/9 A —► future treat
dispenser / servo rail
BUS —► 1/6 divider —► Arduino A0 (battery telemetry, SAFE @3.5 V/cell)
```

Power/endurance budget (nominal patrol): Jetson 10–15 W (cap with `nvpmode1` 15 W mode; Super/25 W only if needed), Pi 5 + peripherals 8–12 W, lidar 2.5 W, OAK-D 4 W, motors avg 5–8 W (peak 40 W stall). Average ≈ 33 –40 W $\rightarrow$ 44 Wh pack $\rightarrow$ **~ 60 –75 min**, meeting SYS-PLT-1 with margin management (throttle Jetson, patrol duty-cycling). Add a low-ESR bulk cap (≥ 1000 μ F) at the Jetson feed to ride through motor-stall sag; keep motor and compute wiring as separate home-runs to the bus (star topology, common ground at bus only).

Protection & safety: individual branch fuses per above; LiPo alarm/telemetry with SAFE mode at 3.5 V/cell (SYS-PLT-2); Arduino hardware watchdog stops PWM if the Jetson heartbeat drops (SYS-PLT-5); master switch reachable on top deck; charge off-board with a balance charger, never on the robot (MVP).

4.3 Physical layout (two-deck chassis suggestion)

- **Lower deck:** battery (center, low CG), L298N, bus/fuses, motors/wheels, caster, Arduino, BNO055 (mount near rotation center, away from motor magnets), 5 V

regulators.

- **Upper deck:** RPLidar A1 (topmost, 360° clear), OAK-D Lite forward at ~0.2–0.3 m height angled slightly down (dog is low!), NoIR + thermal co-located forward, ReSpeaker on top (unobstructed for DoA), Pi 5 + Jetson with airflow, speaker forward-facing.
- Dachshund-specific: sensors aimed for a 0.15–0.25 m-tall subject — tilt cameras ~10–15° downward; keep robot speed/appearance calm (SYS-PLT-6).

5. Software Design

5.1 Stack & repos (open source throughout)

Layer	Choice	Notes
OS	Jetson: JetPack 6 (Ubuntu 22.04). Pi 5: Raspberry Pi OS 64-bit or Ubuntu 24.04	
Middleware	ROS 2 Humble on both (native on Jetson; Docker container on Pi 5 to sidestep OS-version mismatch). CycloneDDS with a static peers list (Wi-Fi multicast is flaky)	Alternative: Jazzy everywhere via containers
SLAM/Localization	<code>slam_toolbox</code> (mapping) → save map → <code>nav2_amcl</code> (runtime) + <code>robot_localization</code> EKF (wheel odom ⊕ BNO055)	
Navigation	Nav2: DWB or MPPI controller, behavior server recoveries (satisfies SYS-NAV-4), waypoint follower, keepout/speed-restricted zones (SYS-NAV-5 via speed-filter mask around detected dog)	
Base control	<code>ros2_control</code> + <code>diffdrive_arduino</code> hardware interface ⊕ <code>ros_arduino_bridge</code> -style firmware on the Nano (PID @ ~30–50 Hz, encoder ticks, IMU, battV in the serial frame)	Well-documented open-source pairing
Camera/DNN	<code>depthai-ros</code> : OAK-D Lite runs YOLOv8n on-device (spatial detection network → 3-D dog position, ~15–20 FPS, zero Jetson GPU load). Jetson GPU stays free for a future finer model (pose/behavior)	

Layer	Choice	Notes
Thermal	<code>mlx90640</code> Python driver → <code>sensor_msgs/Image</code> 32×24 + blob-detector node	
Audio	XVF3800 as USB audio; YAMNet (TFLite) or PANNs-CNN14 classifier node → <code>AudioEvent{class, score, dB, doa}</code> ; DoA read via Sseed host-control API	
Mission logic	BehaviorTree.CPP / nav2_behavior_tree custom BT: Patrol ↔ Investigate ↔ Track/Observe, with <code>PolicyDecision</code> BT node = extension point (MVP: <code>ObserveOnlyPolicy</code> constant)	SYS-EXT-2
Logging	<code>dog_logger</code> node → SQLite (WAL) <code>events(t, state, conf, x, y, room, img_path, context_json, action, outcome)</code> + snapshot JPEGs	SYS-STL-*, SYS-EXT-3
Reporting	<code>daily_report</code> (systemd timer): pandas/Jinja2 → Markdown+HTML with matplotlib timeline → served by a small FastAPI/nginx on the Pi	SYS-RPT-*
Operator UI	Foxglove Studio (or RViz2) on host via router; <code>teleop_twist_keyboard</code> /joystick; web report page	SYS-PLT-4

5.2 ROS 2 node/topic architecture (this table = IBD-03)

Node («rosNode»)	Host	Subscribes	Publishes / serves
<code>base_bridge</code> (ros2_control)	Jetson	<code>/cmd_vel</code>	<code>/odom</code> , <code>/imu/data</code> , <code>/battery_state</code> , <code>/joint_states</code>
<code>rplidar_node</code>	Jetson	—	<code>/scan</code>
<code>ekf_localization</code>	Jetson	<code>/odom</code> , <code>/imu/data</code>	<code>/odometry/filtered</code> , TF
<code>slam_toolbox</code> / <code>amcl</code>	Jetson	<code>/scan</code> , TF	<code>/map</code> , TF map→odom

Node («rosNode»)	Host	Subscribes	Publishes / serves
<code>nav2</code> stack	Jetson	<code>/map</code> , <code>/scan</code> , <code>/odometry/filtered</code>	<code>/cmd_vel</code> ; actions <code>NavigateToPose</code> , <code>FollowWaypoints</code>
<code>oakd_dog_detector</code>	Jetson(+OAK)	—	<code>/dog/detections_3d</code> (class, conf, xyz), <code>/oak/rgb/preview</code>
<code>dog_locator</code>	Jetson	<code>/dog/detections_3d</code> , TF	<code>/dog/pose_map</code> , <code>/dog/found</code>
<code>mission_bt</code>	Jetson	<code>/dog/*</code> , <code>/audio/events</code> , <code>/battery_state</code>	mode status; calls Nav2 actions + engagement action servers
<code>engage_actions</code> (ApproachDog, Retreat, Speak, DispenseTreatStub)	Jetson/Pi	goals from BT	action feedback; uniform interface for future policy (SYS-EXT-1)
<code>thermal_node</code> + <code>thermal_blob</code>	Pi	—	<code>/thermal/image</code> , <code>/thermal/blob</code>
<code>noir_cam</code>	Pi	—	<code>/noir/image</code> (night mode)
<code>audio_classifier</code>	Pi	(XVF3800 stream)	<code>/audio/events</code> (bark/whine/loud + DoA)
<code>state_fusion</code>	Pi	<code>/dog/pose_map</code> , <code>/dog/detections_3d</code> , <code>/thermal/blob</code> , <code>/audio/events</code>	<code>/billie/state</code> (state, conf, <code>context[]</code> , <code>stress_proxy</code>)
<code>dog_logger</code>	Pi	<code>/billie/state</code> , <code>/dog/found</code> , action results	SQLite writes; <code>/events/last</code>
<code>daily_report</code>	Pi	(timer)	HTML/MD report on <code>http://pi:8080/report</code>
<code>speaker_node</code>	Pi	<code>/speak</code> (rate-limited)	I ² S audio out

5.3 State classification (MVP heuristic, ML-ready interface)

`state_fusion` maintains a 10 s sliding window and applies hysteresis:

- **BARKING** — `audio.class ∈ {bark,howl,whine}` with score $> \theta$ (DoA optionally gated to dog bearing).
- **ACTIVE** — visual motion energy of the dog bbox $> \theta_m$, or dog map-position displacement $> 0.3 \text{ m}/10 \text{ s}$.
- **SLEEPING** — detection present, bbox aspect ratio flat ($w/h > 2$ for a dachshund lying down!), motion ≈ 0 for $> 120 \text{ s}$, thermal blob static; **RESTING** = same but $< 120 \text{ s}$ or intermittent motion.
- **NOT-FOUND** — no fused evidence for $> N$ minutes.

The node's I/O contract (`/billie/state` with a raw `context[]` feature vector) is exactly what a learned classifier — and later the contextual bandit — will consume, so upgrading the internals requires no interface change (SYS-STL-3).

5.4 Behavior-AI insertion plan (design-for, not build)

- **Where:** the `PolicyDecision` BT node inside `mission_bt`. MVP implementation returns `OBSERVE`. v2 swaps in `BanditPolicy` (e.g., LinUCB/Thompson over `context[]` from `/billie/state`) choosing among `{OBSERVE, APPROACH, RETREAT, SPEAK, TREAT}` — all of which already exist as action servers (SYS-EXT-1).
- **Reward plumbing:** `dog_logger` already records `(context, action, outcome)`; outcome features = Δ engagement (dog approaches/orients toward robot, from `/dog/pose_map` relative motion) and `stress_proxy` penalty (bark rate spike, rapid retreat). MVP data collected under `ObserveOnlyPolicy` provides the baseline distribution.
- **Safety rails carried into v2:** standoff floor, speed caps near dog, speaker rate limits, and per-action cooldowns are enforced *below* the policy (in the action servers), so no learned policy can violate them.

6. Build, Integration & Verification Plan

Phase	Content	Exit criteria (→ req)
0 Bench	Power bus + fusing; Arduino firmware (PID, encoders, IMU, battV, heartbeat cutoff); serial protocol soak	Motors hold commanded velocity $\pm 10 \%$; heartbeat cutoff $< 500 \text{ ms}$ (TC-PLT-5)

Phase	Content	Exit criteria (→ req)
1 Teleop	ROS 2 up on Jetson; <code>ros2_control</code> diff drive; teleop from host via router	Smooth teleop 15 min; odom drift characterized (SYS-PLT-4)
2 Mapping	RPLidar + slam_toolbox; drive apartment; save map; AMCL relocalization	Map complete; reloc $\leq$ 0.15 m (TC-NAV-2)
3 Autonomy	Nav2 tuning (footprint, DWB/MPPI, recoveries); patrol waypoints; stuck-recovery tests (rug edges, chair legs!)	10 patrol laps, 0 human interventions (TC-NAV-3/4/6)
4 Perception	OAK-D YOLO dog detection + 3-D locate; thermal blob; audio classifier on recorded Billie barks; NoIR/night trial	TC-PER-1...5 pass thresholds
5 Mission	Mission BT: patrol→investigate→track/observe; state fusion; logger; standoff behavior with the real dog (short, calm sessions)	Finds Billie $\leq$ 10 min in $\geq$ 8/10 trials (TC-FND-4); correct state labels on a 1 h annotated session (TC-STL-1)
6 MVP acceptance	24 h soak: scheduled patrols, event logging, daily report at 23:30, SAFE-mode battery test	All SYS-RPT, SYS-PLT tests pass; report reviewed by operator

Known gaps / purchase-nothing-yet risks: IR illuminator for NoIR (or accept thermal-only at night); L298N thermal margins (heatsink now, driver swap later); Wi-Fi DDS discovery (use CycloneDDS unicast peers or a Zenoh bridge if flaky); Nano flash headroom (keep firmware lean — no micro-ROS on the 328P, custom serial only).

Appendix A — Cameo modeling checklist (quick start)

1. New project → SysML; create the package tree of §3.1; add the tiny profile with «futureRelease», «rosNode», «rosTopic».
2. REQ-01/REQ-02 per §2.3 (import the requirement tables via Excel/CSV sync to save typing).
3. BDD-00...BDD-06 per §3.2–3.4 (blocks + composition; add value properties from the tables).
4. Interface blocks & signals (§3.5 item-flow lists) in package 5, then draw IBD-00...IBD-03.

5. STM-01 + ACT-01...ACT-04 per §3.6 with partitions allocated to subsystem blocks.
6. Generate satisfy/allocation matrices in package 6; confirm every SYS-* has ≥ 1 satisfying block and ≥ 1 verifying test case.